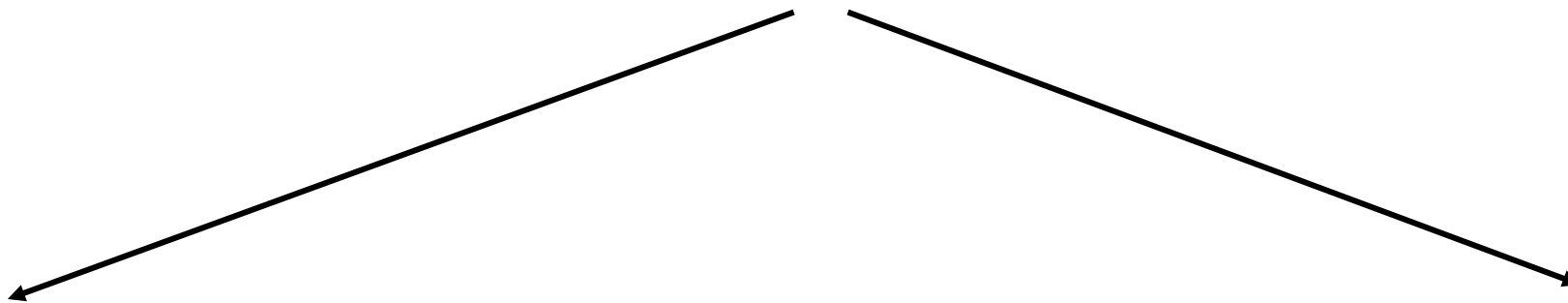




Ассоциативные контейнеры STL



Контейнеры STL



Последовательные

array
vector
deque
forward_list
list

Ассоциативные

set
map
multiset
multimap
unordered_map
unordered_set



Ассоциативный массив



Ассоциативный массив — абстрактный тип данных, позволяющий хранить пары вида «(ключ, значение)» и поддерживающий операции **добавления** пары, а также **поиска** и **удаления** пары по ключу.



Std::map

```
#include <iostream>
#include <map>
#include <string>

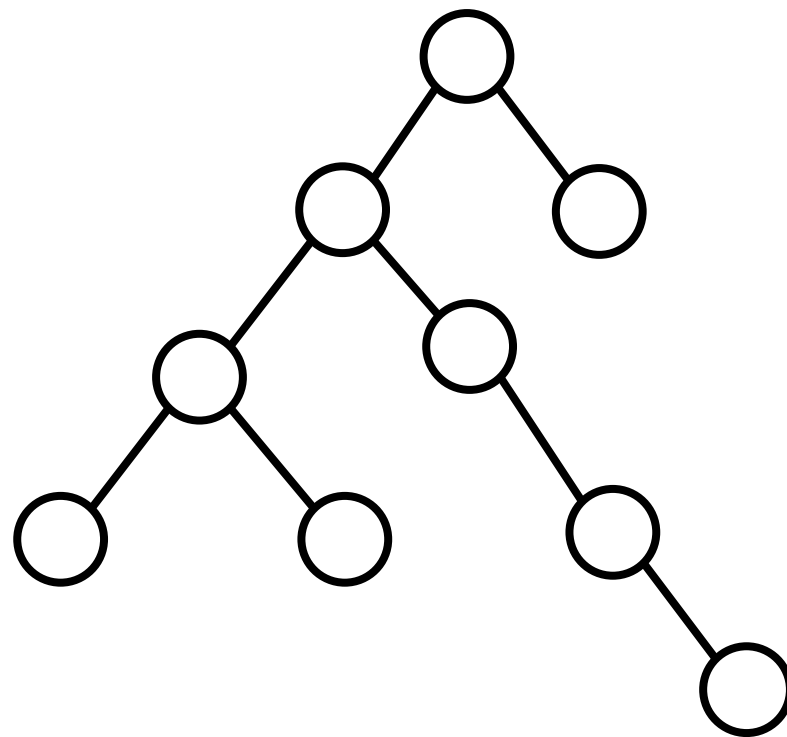
int main()
{
    std::map<std::string, int> m;
    m["Math"] = 5;
    m["Literature"] = 4;
    m["Russian"] = 4;
    print_map(m);
}
```

Отсортированный ассоциативный контейнер, позволяющий хранить пары [ключ — значение]. Значение ключа — уникально.

Полный код см. в репозитории



Деревья



Бинарное дерево — это структура данных, в которой каждый узел имеет не более двух потомков

Корень — верхний узел дерева, у которого нет родителя.

Листовой узел (терминальный) — узел, у которого нет потомков (оба указателя NULL)

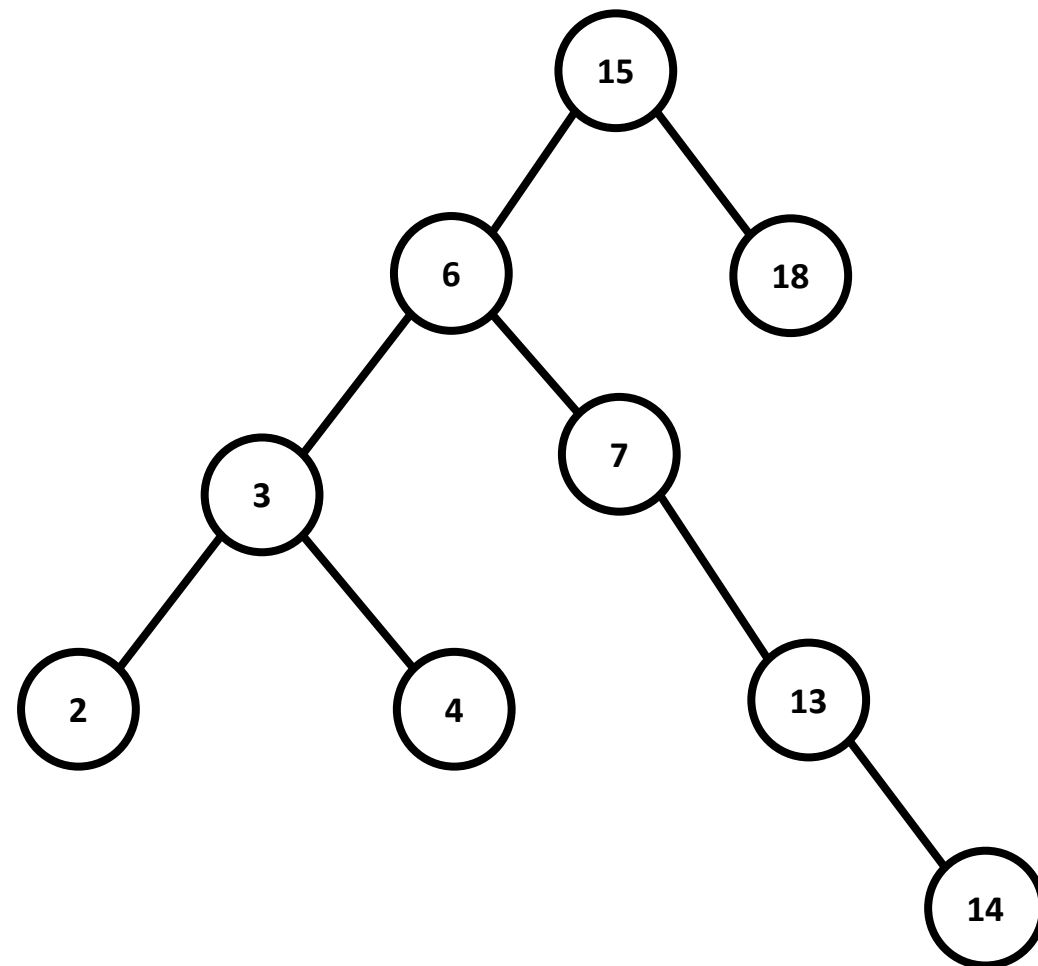
Высота узла — это длина самого длинного пути от этого узла до листа.



Бинарное дерево поиска: поиск

ITERATIVE_TREE_SEARCH(x, k)

```
1 while  $x = \text{NIL}$  и  $k = \text{key}[x]$   
2   do if  $k < \text{key}[x]$   
3     then  $x \leftarrow \text{left}[x]$   
4     else  $x \leftarrow \text{right}[x]$   
5 return  $x$ 
```





Бинарное дерево поиска: поиск

TREE_SEARCH(x, k)

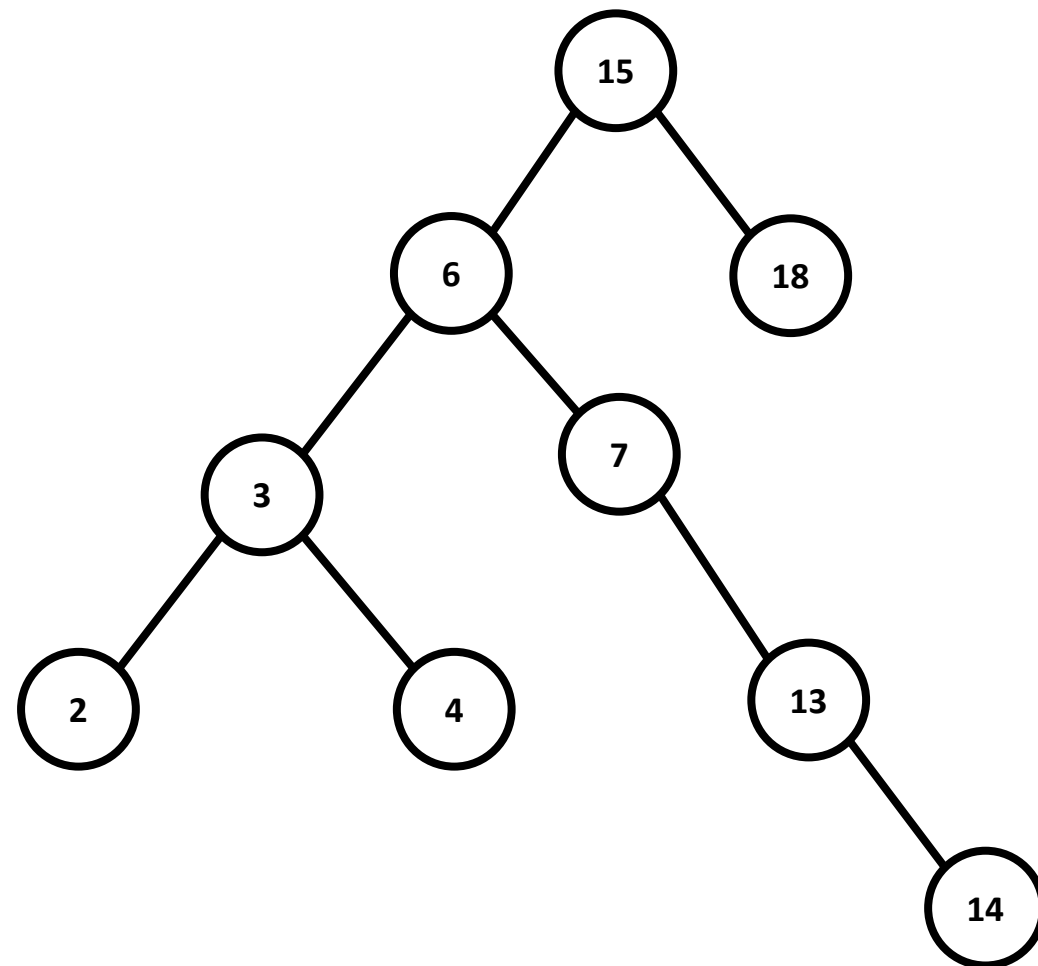
1 **if** $x = \text{NIL}$ или $k = \text{key}[x]$

2 **then return** x

3 **if** $k < \text{key}[x]$

4 **then return** TREE_SEARCH($\text{left}[x], k$)

5 **else return** TREE_SEARCH($\text{right}[x], k$)





Бинарное дерево поиска: вставка

TREE_INSERT(T, z)

1 $y \leftarrow \text{NIL}$

2 $x \leftarrow \text{root}[T]$

3 **while** $x = \text{NIL}$

4 **do** $y \leftarrow x$

5 **if** $\text{key}[z] < \text{key}[x]$

6 **then** $x \leftarrow \text{left}[x]$

7 **else** $x \leftarrow \text{right}[x]$

8 $p[z] \leftarrow y$

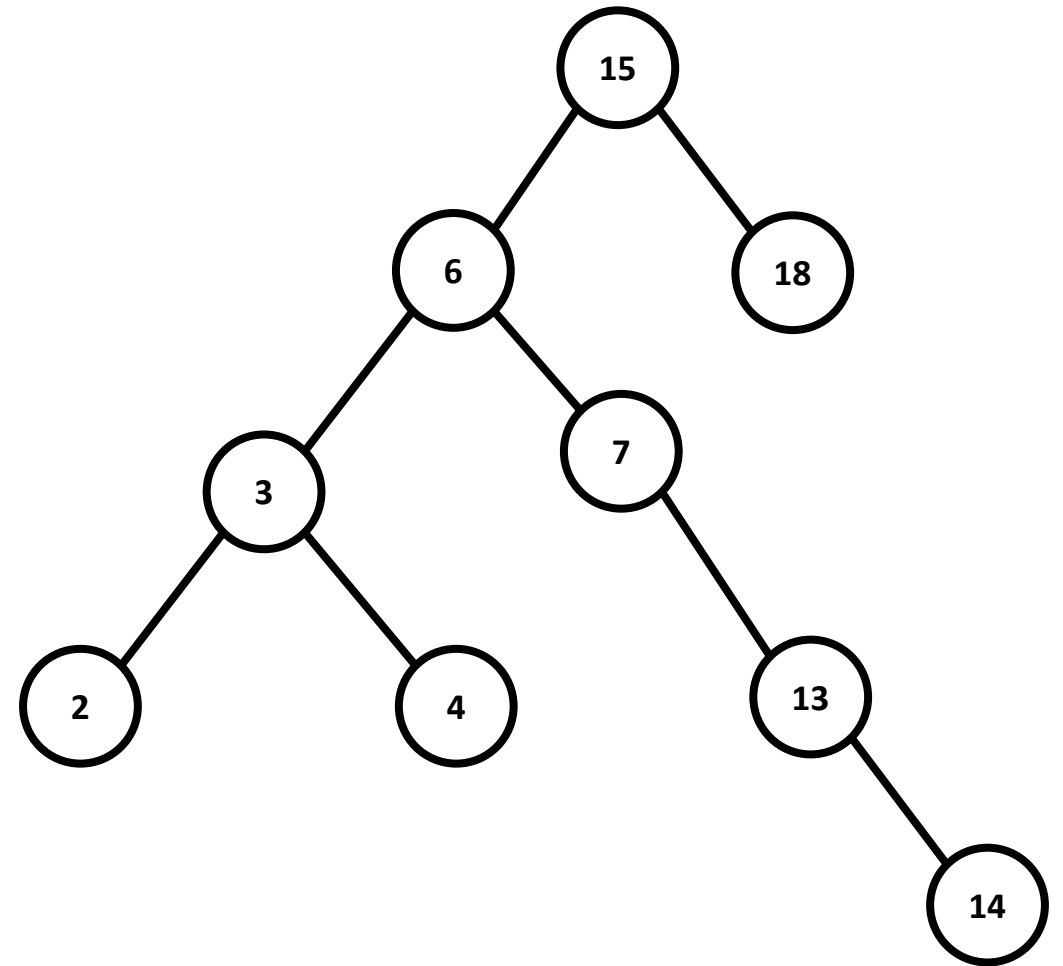
9 **if** $y = \text{NIL}$

10 **then** $\text{root}[T] \leftarrow z$ Дерево T — пустое

11 **else if** $\text{key}[z] < \text{key}[y]$

12 **then** $\text{left}[y] \leftarrow z$

13 **else** $\text{right}[y] \leftarrow z$





Бинарное дерево поиска: вставка

TREE_INSERT_RECURSIVE(x, z)

1 **if** x = NIL

2 **then** return z

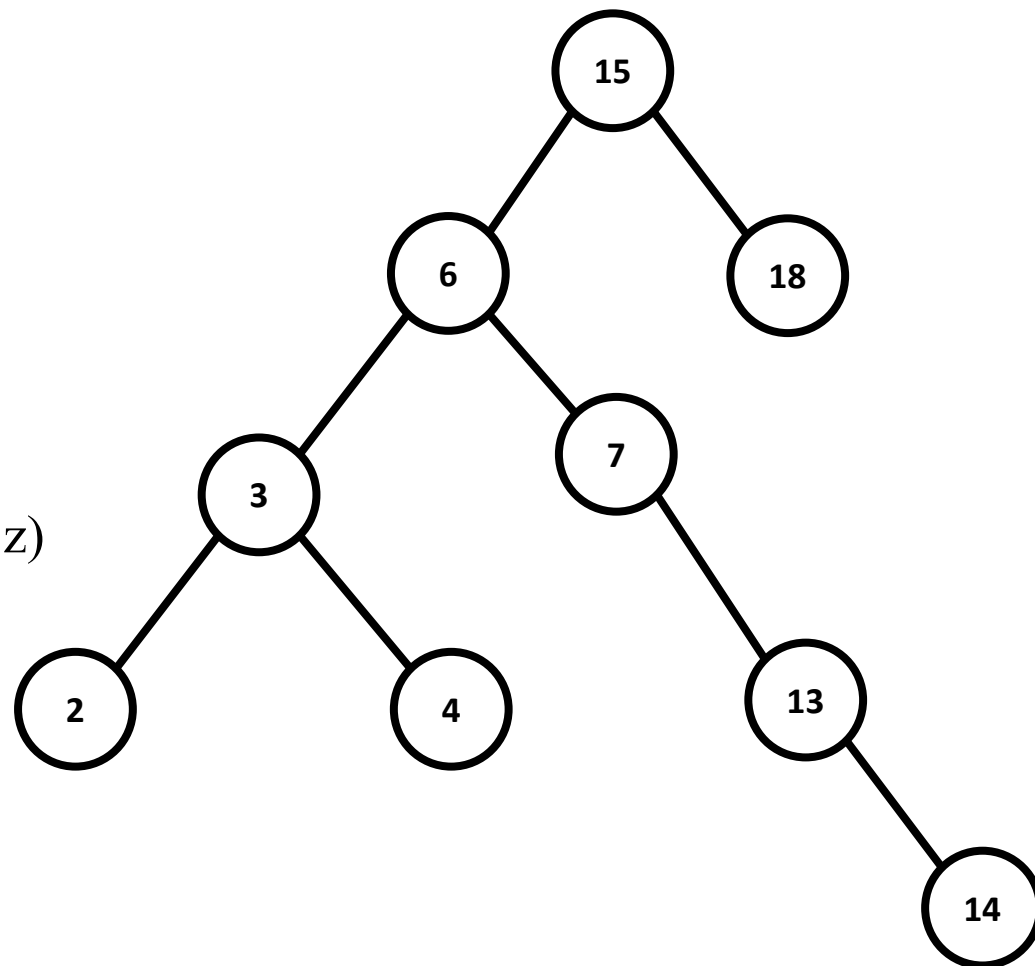
3 **if** key[z] < key[x]

4 **then** left[x] $\leftarrow$ TREE_INSERT_RECURSIVE(left[x], z)

5 **else**

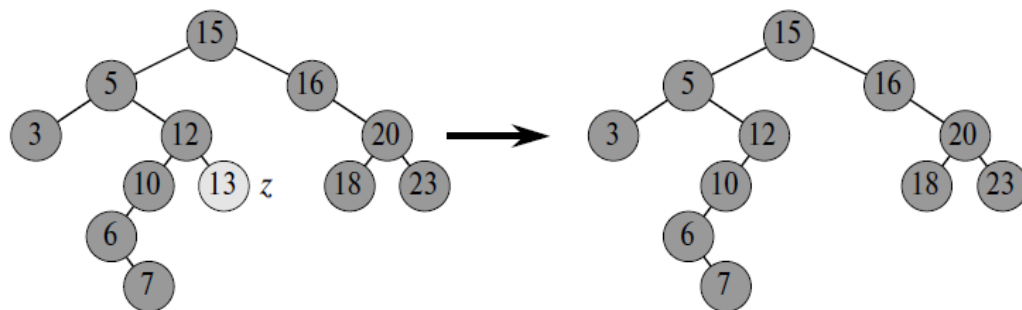
6 right[x] $\leftarrow$ TREE_INSERT_RECURSIVE(right[x], z)

7 **return** x

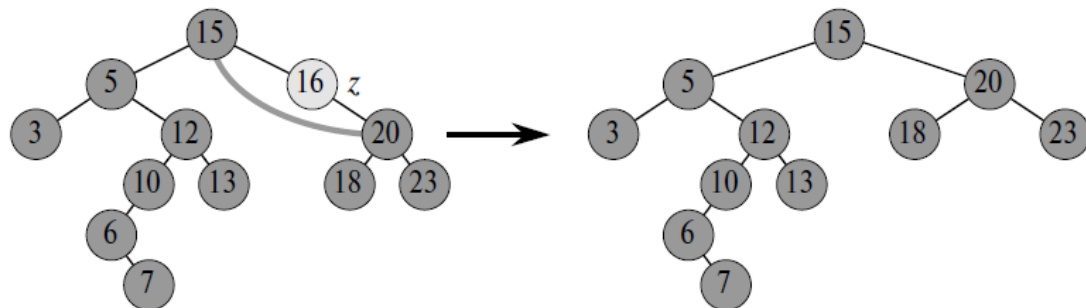




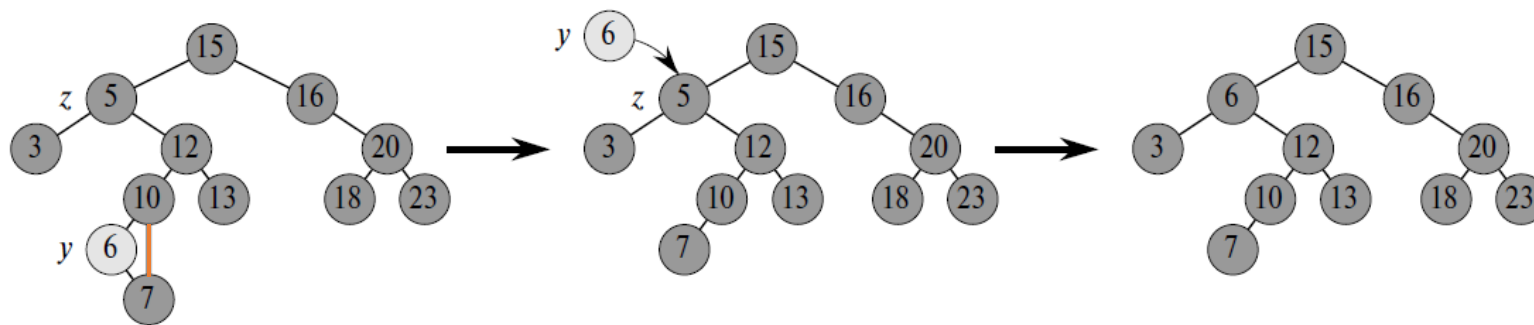
Бинарное дерево поиска: удаление



a)



б)



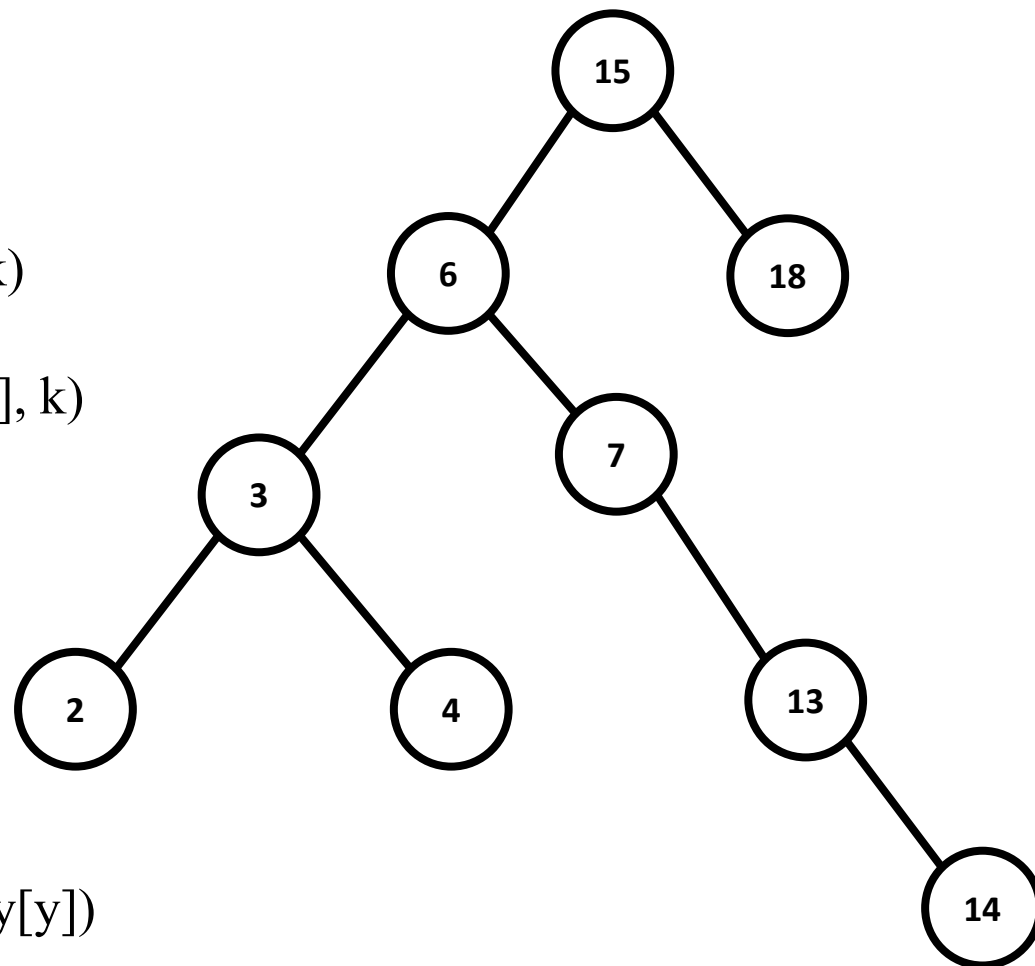
в)



Бинарное дерево поиска: удаление

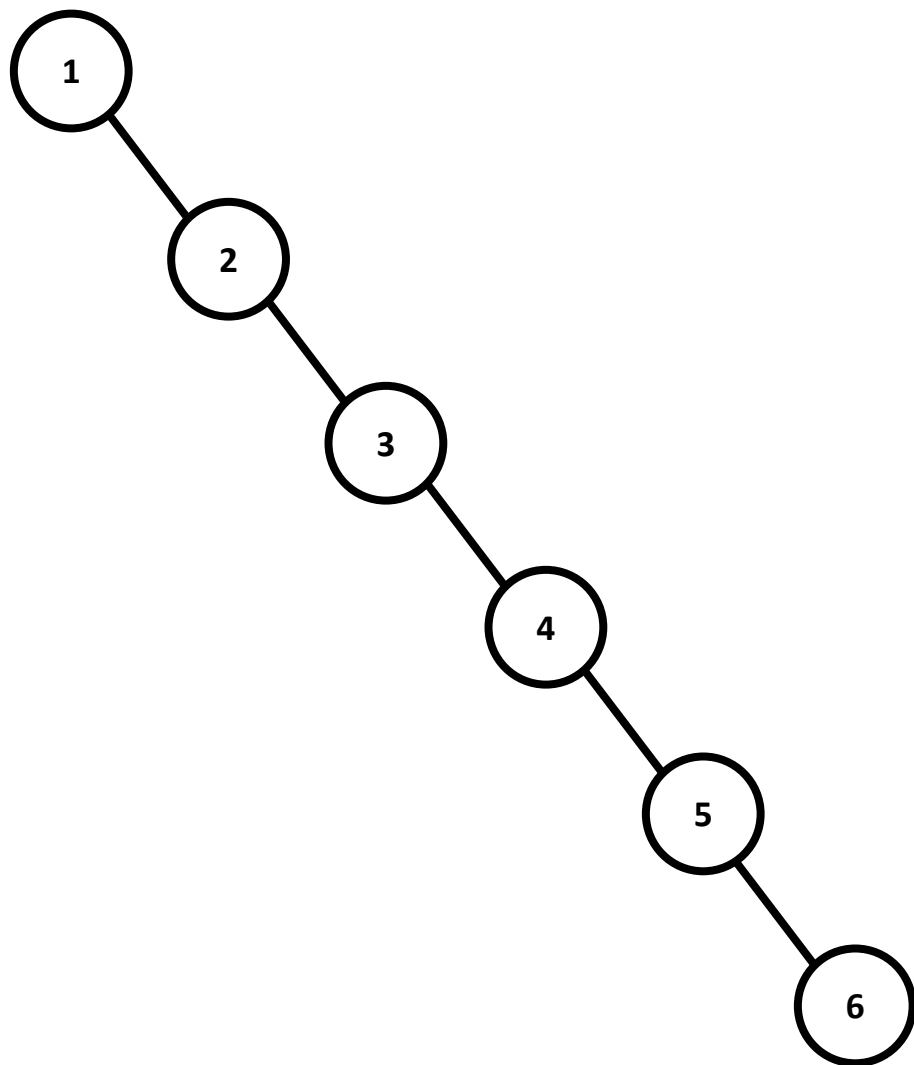
TREE_DELETE_RECURSIVE(x, k)

```
1 if x = NIL
2   then return x
3 if k < key[x]
4   then left[x] ← TREE_DELETE_RECURSIVE(left[x], k)
5 else if k > key[x]
6   then right[x] ← TREE_DELETE_RECURSIVE(right[x], k)
7 else
8   if left[x] = NIL
9     then return right[x]
10  else if right[x] = NIL
11    then return left[x]
12  y ← TREE_MINIMUM(right[x])
13  key[x] ← key[y]
14  right[x] ← TREE_DELETE_RECURSIVE(right[x], key[y])
15 return x
```



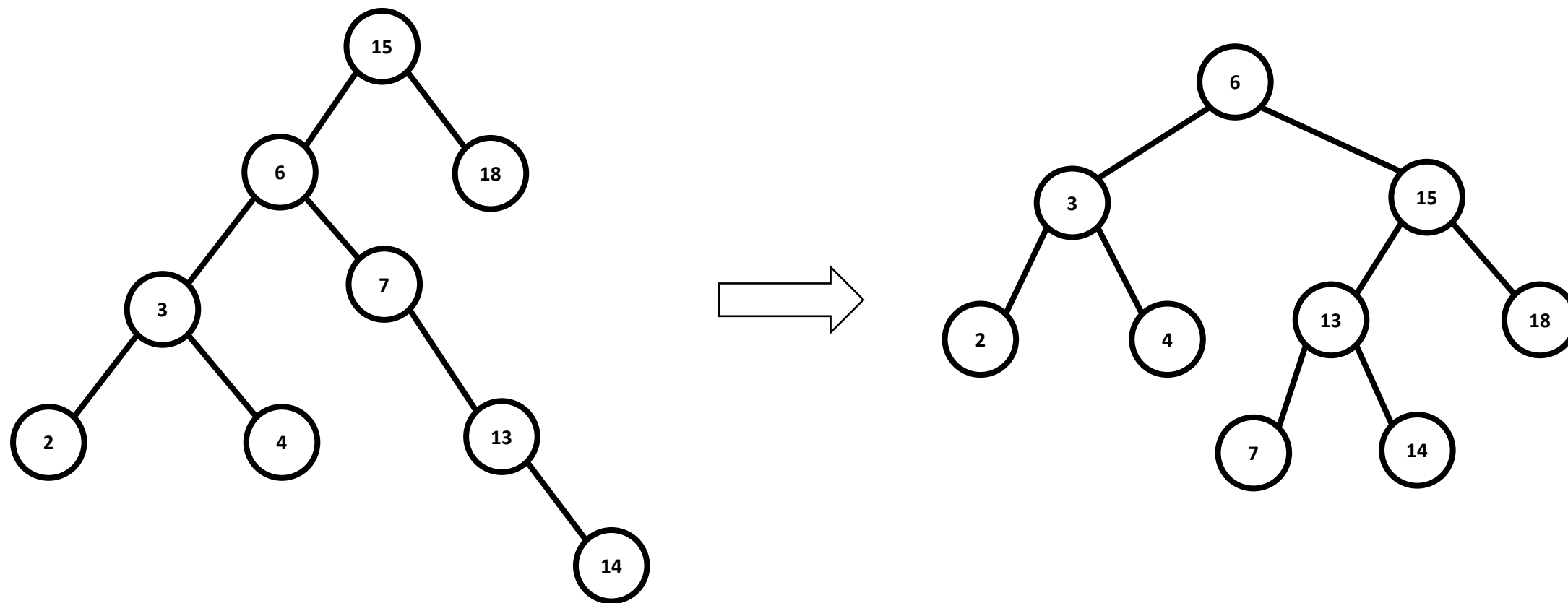


И жили они долго и счастливо (нет)





Сбалансированные деревья



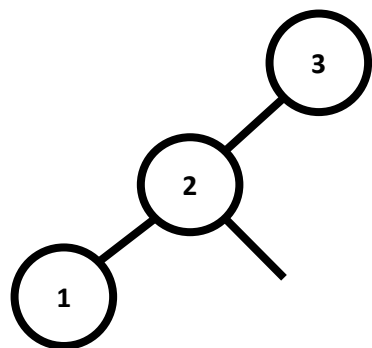
AVL-дерево — это бинарное дерево поиска, в котором для любого узла выполняется условие:
 $|h(\text{left}) - h(\text{right})| \leq 1$

где:

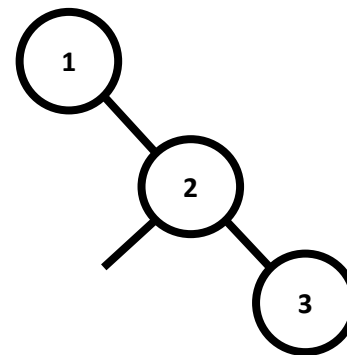
- $h(\text{left})$ — высота левого поддерева
- $h(\text{right})$ — высота правого поддерева



Повороты деревьев



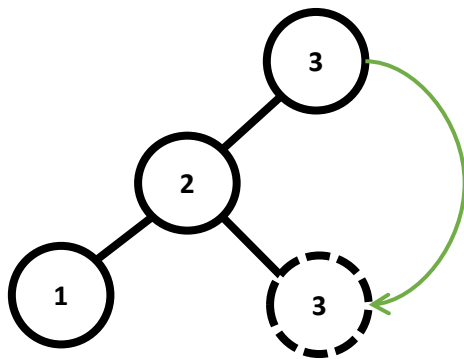
Правый поворот



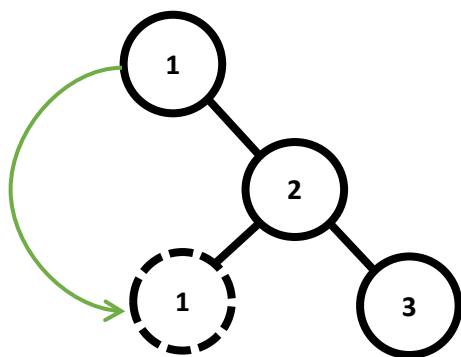
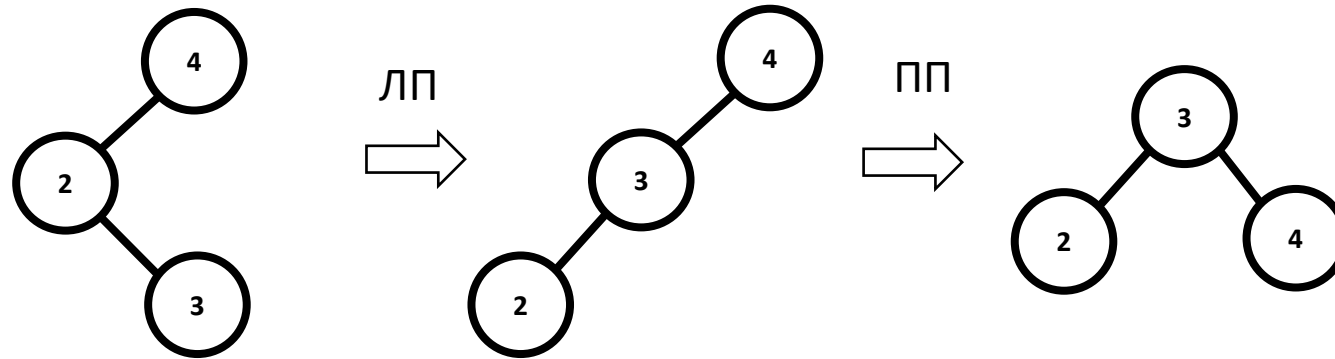
Левый поворот



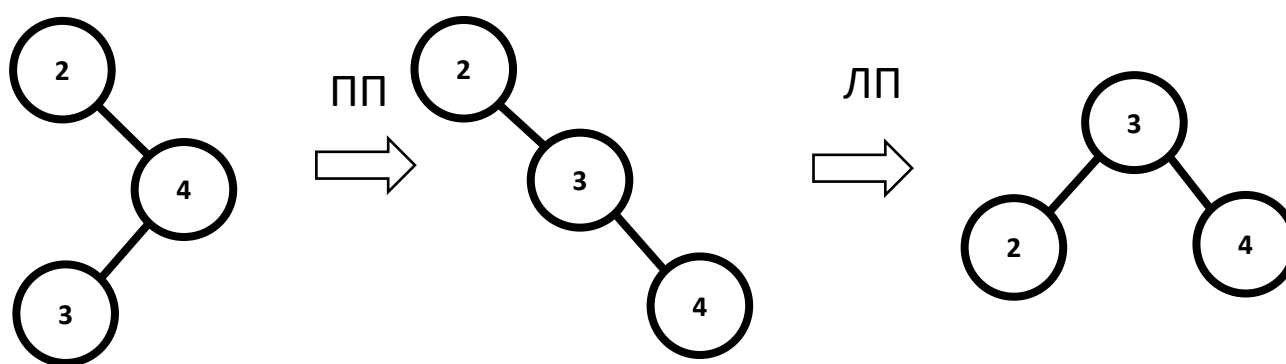
Повороты деревьев



Правый поворот

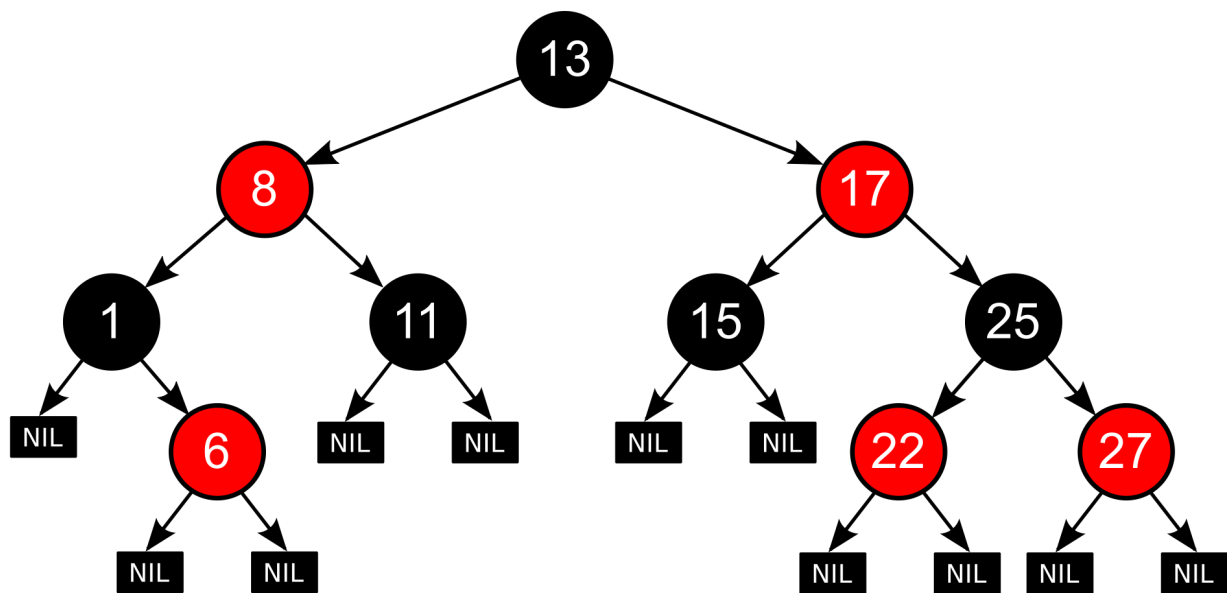


Левый поворот





Красно-черное дерево: реализация



1. Узел может быть либо красным, либо чёрным и имеет двух потомков;
2. Корень — как правило чёрный. Это правило слабо влияет на работоспособность модели, так как цвет корня всегда можно изменить с красного на чёрный;
3. Все листья, не содержащие данных — чёрные.
4. Оба потомка каждого красного узла — чёрные.
5. Любой простой путь от узла-предка до листового узла-потомка содержит одинаковое число чёрных узлов.

Красно-чёрное дерево — двоичное дерево поиска, в котором каждый узел имеет атрибут *цвета*.

Полный код см. в репозитории



Std::unordered_map

```
#include <iostream>
#include <unordered_map>
#include <string>

int main()
{
    std::unordered_map<std::string, int> m;
    m["Math"] = 5;
    m["Literature"] = 4;
    m["Russian"] = 4;
    print_map(m);
}
```

Ассоциативный контейнер, позволяющий хранить пары [ключ — значение]. Представлен в виде хэш-таблицы

Полный код см. в репозитории

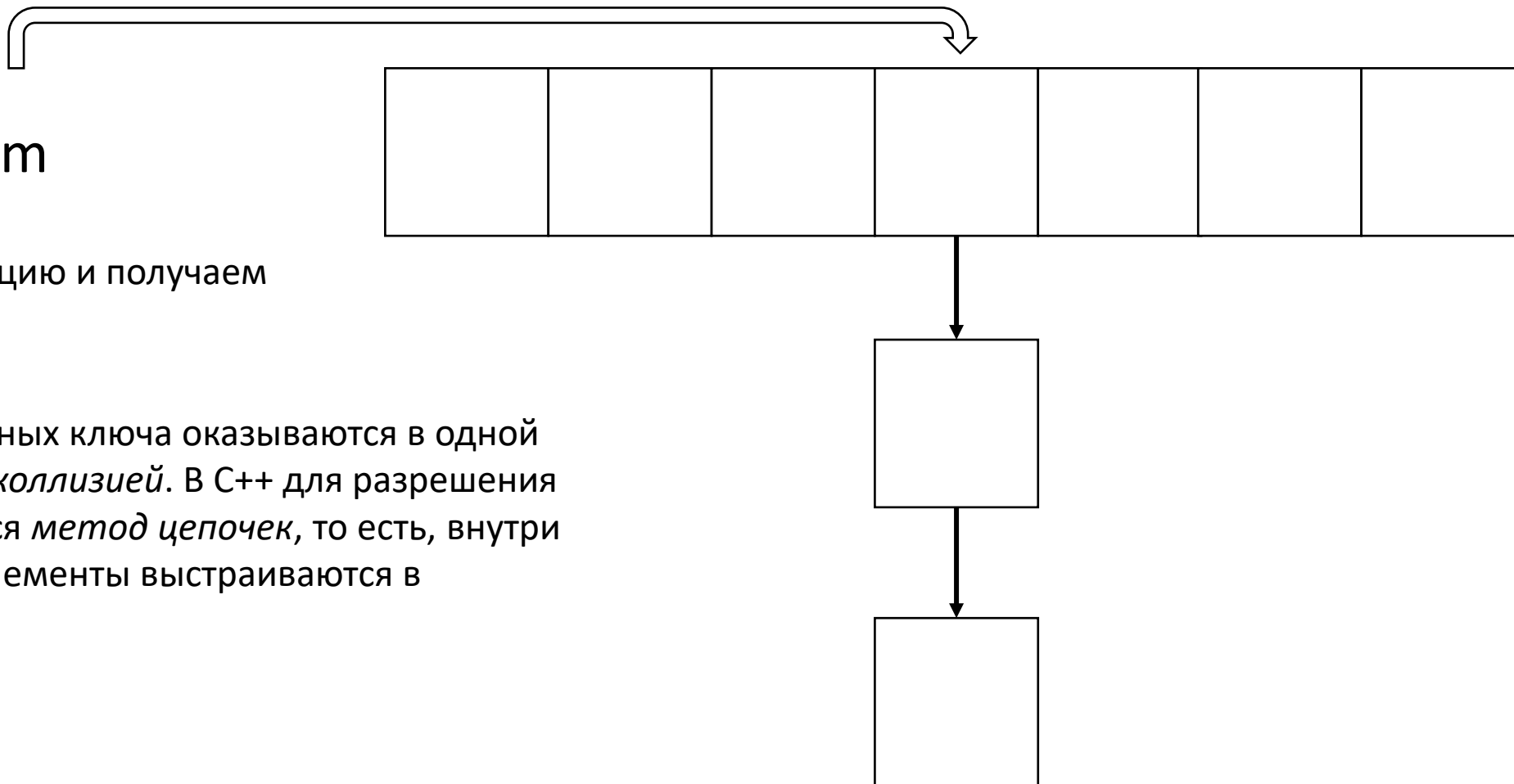


Хэш-таблица: реализация

Hash: key $\rightarrow$ num

Применяем хэш функцию и получаем уникальное число.

Случай, когда два разных ключа оказываются в одной корзине, называется *коллизией*. В C++ для разрешения коллизий используется *метод цепочек*, то есть, внутри одной корзины все элементы выстраиваются в односвязный список.



Полный код см. в репозитории



Std::multimap

```
#include <iostream>
#include <map>
#include <string>
```

```
int main()
{
    std::multimap<std::string, int> m;
    m.insert(std::pair<std::string, int>("Math", 5));
    m.insert(std::pair<std::string, int>("Math", 4));
    m.insert(std::pair<std::string, int>("Russian", 5));
    print_map(m);
    m.erase("Math");
    print_map(m);
}
```

Отсортированный ассоциативный контейнер, позволяющий хранить пары [ключ — значение]. Значение ключа — может быть не уникальным.

Полный код см. в репозитории



Std::set

```
#include <iostream>
#include <set>
```

```
int main()
{
    std::set<int> s;
    s.insert(5);
    s.insert(34);
    s.insert(321);
    s.insert(5);
    print_set(s);
}
```

set — это контейнер, который автоматически сортирует добавляемые элементы в порядке возрастания. Но при добавлении одинаковых значений, set будет хранить только один его экземпляр. По другому его еще называют множеством.

Полный код см. в репозитории



Std::multiset

```
#include <iostream>
#include <map>
```

```
int main()
{
    std::multiset<int> s;
    s.insert(5);
    s.insert(34);
    s.insert(321);
    s.insert(5);
    print_set(s);
}
```

multiset — это контейнер, который будет содержать элементы в отсортированном порядке при добавлении, но он хранит повторяющиеся элементы, по сравнению с множеством set.

Полный код см. в репозитории



Зачем все это нужно?

Container	Insert Head	Insert Tail	Insert	Remove Head	Remove Tail	Remove	Index Search	Find
vector	n/a	$O(1)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(1)$	$O(\log n)$
list	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	n/a	$O(n)$
deque	$O(1)$	$O(1)$	n/a	$O(1)$	$O(1)$	$O(n)$	n/a	n/a
queue	n/a	$O(1)$	n/a	$O(1)$	n/a	n/a	$O(1)$	$O(\log n)$
stack	$O(1)$	n/a	n/a	$O(1)$	n/a	n/a	n/a	n/a
map	n/a	n/a	$O(\log n)$	n/a	n/a	$O(\log n)$	$O(1)$	$O(\log n)$
multimap	n/a	n/a	$O(\log n)$	n/a	n/a	$O(\log n)$	$O(1)^*$	$O(\log n)$
set	n/a	n/a	$O(\log n)$	n/a	n/a	$O(\log n)$	$O(1)$	$O(\log n)$
multiset	n/a	n/a	$O(\log n)$	n/a	n/a	$O(\log n)$	$O(1)^*$	$O(\log n)$



Сложность алгоритмов

	vector	deque	list	set	map
operator[i]	$O(1)$	$O(1)$	-	-	$O(\log(n))$
insert(iter, val)	$O(n)$	$O(1)$	$O(1)$	$O(\log(n))$	$O(\log(n))$
erase(iter)	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
push_back(val)	$O(1)$	$O(1)$	$O(1)$	-	-
push_front(val)	-	$O(1)$	$O(1)$	-	-
find	-	-	-	$O(\log(n))$	$O(\log(n))$



Сложность алгоритмов

$O(1)$ – для операции требуется константное время

$O(n)$ – полный перебор

$O(\log(n))$ – бинарный поиск

$O(n^2)$ – сортировка пузырьком (двойной полный перебор)